

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»
(Университет ИТМО)**

КУРСОВАЯ РАБОТА

«Разработка Android-приложения “Мессенджер”»

Выполнила:

Невзорова Вероника, К3443

Преподаватель:

Шатинский Григорий Сергеевич

Санкт-Петербург
2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	3
1. Архитектура приложения и навигация.....	5
2. Управление состоянием и архитектура MVVM.....	7
3. Работа с данными и офлайн-доступ.....	10
4. Улучшение интерфейса и фоновая синхронизация.....	12
ЗАКЛЮЧЕНИЕ.....	14

ВВЕДЕНИЕ

Мобильные приложения занимают центральное место в современном цифровом пространстве: миллиарды пользователей ежедневно взаимодействуют с приложениями на платформе Android, которая уверенно удерживает лидерство на мировом рынке мобильных операционных систем. Разработка качественных Android-приложений требует от специалиста не только навыков создания пользовательского интерфейса, но и глубокого понимания архитектурных подходов, механизмов управления состоянием, работы с данными и системными сервисами платформы.

Целью настоящей курсовой работы является разработка Android-приложения «Мессенджер», включающего функционал просмотра ленты сообщений, управления профилем пользователя и настройки параметров приложения. Проект реализован поэтапно в ходе выполнения четырёх лабораторных работ, каждая из которых последовательно наращивала функциональность и архитектурную зрелость системы.

Для достижения поставленной цели были сформулированы следующие задачи: спроектировать и реализовать структуру приложения на основе Activity и Fragment с использованием встроенной навигации Android; внедрить архитектурный паттерн MVVM для управления состоянием экранов и реактивного обновления интерфейса; организовать загрузку данных из сетевого API и их кэширование в локальной базе данных Room для обеспечения работоспособности приложения в офлайн-режиме; улучшить интерфейс с применением компонентов Material Design; настроить фоновую синхронизацию данных средствами WorkManager и реализовать систему уведомлений.

Реализованное приложение обеспечивает навигацию между основными разделами через нижнюю панель, корректно сохраняет пользовательские данные при смене конфигурации устройства, поддерживает оффлайн-режим и выполняет периодическое обновление данных в фоне. Вся архитектура

приложения строится на принципах разделения ответственности, что обеспечивает удобство сопровождения и дальнейшего развития проекта.

1. Архитектура приложения и навигация

Первым этапом разработки стало создание базовой структуры приложения «Мессенджер». В качестве точки входа была реализована одна главная активность (MainActivity), которая служит контейнером для всей навигационной структуры приложения. Такой подход соответствует концепции Single-Activity Architecture, рекомендованной командой Android и позволяющей централизованно управлять переходами между экранами.

Навигация между разделами приложения реализована с помощью BottomNavigationView — нижней панели с тремя вкладками: «Лента», «Профиль» и «Настройки». Для управления переходами использованы стандартные инструменты Android Navigation Component: NavController и Navigation Graph. NavController отвечает за переключение между фрагментами, а граф навигации декларативно описывает доступные экраны и связи между ними. Это обеспечивает централизованное управление маршрутами и упрощает поддержку приложения.

Каждая вкладка нижней панели открывает отдельный фрагмент. Экран «Лента» на первом этапе реализован в виде заглушки с текстовым содержимым — его функциональность будет расширена в последующих лабораторных работах. Экран «Профиль» отображает основные данные пользователя: имя и статус. Экран «Настройки» включает базовый элемент управления — переключатель для смены темы приложения.

В рамках контроля корректности работы компонентов было реализовано логирование жизненного цикла. В методах onCreate, onStart, onStop и onDestroy для Activity, а также в соответствующих методах каждого Fragment выводятся сообщения в лог с указанием имени компонента. Это позволяет отслеживать порядок создания и завершения работы компонентов в процессе разработки и тестирования, а также лучше понимать поведение системы при смене конфигурации или сворачивании приложения.

Архитектурная основа, заложенная в первой лабораторной работе, обеспечила возможность последовательного расширения функциональности

приложения без необходимости пересмотра базовой структуры. Применение Navigation Component, четкое разграничение экранов через фрагменты и соблюдение принципа единой точки входа создали устойчивый каркас, на котором строились все последующие этапы разработки.

2. Управление состоянием и архитектура MVVM

На втором этапе разработки в проект была внедрена архитектура MVVM (Model–View–ViewModel), которая является рекомендованным подходом к построению Android-приложений. Паттерн обеспечивает четкое разделение ответственности: слой View отвечает исключительно за отображение данных, ViewModel хранит и обрабатывает состояние экранов, а Model представляет источники данных и бизнес-логику. Такое разграничение повышает тестируемость, снижает связность компонентов и упрощает поддержку проекта.

Для экрана «Профиль» была создана ProfileViewModel, хранящая имя и статус пользователя. Пользователь может редактировать эти данные через поля EditText: каждое изменение сохраняется во ViewModel и немедленно отражается в интерфейсе. Для реактивного обновления UI применяется LiveData — при изменении значений во ViewModel подписанные компоненты View автоматически получают актуальные данные. Благодаря тому что ViewModel не привязана к жизненному циклу фрагмента, данные сохраняются при повороте экрана и других изменениях конфигурации устройства.

Аналогичным образом была реализована SettingsViewModel для экрана «Настройки». Состояние переключателя темы (светлая или тёмная) хранится во ViewModel и восстанавливается при пересоздании экрана. При изменении состояния переключателя ViewModel обновляет значение, а View применяет соответствующую тему приложения. Такой подход исключает потерю пользовательских настроек при изменении конфигурации и обеспечивает предсказуемое поведение интерфейса.

Жизненный цикл ViewModel также логируется в рамках требований к наблюдаемости приложения. В методе onCleared(), вызываемом при уничтожении ViewModel, фиксируется соответствующая запись в лог. Это позволяет отслеживать момент освобождения ресурсов и убедиться в корректной работе компонента в различных сценариях взаимодействия с

приложением.

Использование архитектуры MVVM в совокупности с LiveData позволило выстроить однонаправленный поток данных: ViewModel является единственным источником истины для UI, а View лишь наблюдает за изменениями и отображает актуальное состояние. Это значительно упростило логику фрагментов, избавив их от необходимости управлять состоянием самостоятельно, и подготовило основу для подключения реальных источников данных на следующем этапе разработки.

взаимодействие реализовано с помощью Retrofit и корутин, что обеспечивает не блокирующую загрузку данных и корректную обработку ошибок. Таким образом, приложение корректно работает как при наличии подключения к сети, так и при его отсутствии.

Взаимодействие между слоями приложения организовано таким образом, что изменения данных автоматически отражаются в пользовательском интерфейсе без необходимости ручного обновления. ViewModel выступает связующим звеном между UI и источниками данных, обеспечивая единый поток данных и предсказуемое поведение приложения. Использование архитектуры MVVM в совокупности с современными инструментами Android позволило создать устойчивую и расширяемую архитектуру, соответствующую актуальным рекомендациям платформы.

3. Работа с данными и офлайн-доступ

Третья лабораторная работа была посвящена реализации полноценного слоя данных: получению списка сообщений из удаленного API и обеспечению их доступности в отсутствие сети через локальное хранилище. Главной задачей этапа стало построение надежного контура данных, при котором экран «Лента» всегда отображает актуальную информацию независимо от состояния интернет-соединения.

Сетевое взаимодействие реализовано с использованием библиотеки Retrofit. Для взаимодействия с публичным API создан интерфейс сервиса, объявляющий suspend-функции для выполнения HTTP-запросов. Использование suspend-функций позволяет запускать сетевые операции в контексте корутин: они выполняются асинхронно и не блокируют главный поток, что обеспечивает отзывчивость пользовательского интерфейса даже при медленном соединении. Конфигурация Retrofit включает базовый URL и конвертер Gson для автоматического разбора JSON-ответов в Kotlin-объекты.

Для локального хранения данных была подключена база данных Room. Структура локального хранилища включает три ключевых компонента: Entity — класс, описывающий таблицу сообщений (идентификатор, имя автора, текст и дополнительные поля); DAO — интерфейс с методами для вставки, обновления и выборки записей; Database — точка инициализации Room, объединяющая все DAO проекта. Методы DAO, предназначенные для чтения данных, возвращают Flow, что позволяет UI автоматически получать обновления при изменении содержимого базы данных без дополнительных ручных вызовов.

Для инкапсуляции логики работы с данными создан MessageRepository — единая точка взаимодействия ViewModel с источниками данных. Репозиторий определяет стратегию получения данных: при наличии сети выполняется загрузка из API с последующим сохранением результата в Room; при отсутствии соединения возвращаются данные из локальной базы.

Результат операции обновления оборачивается в объект Result для явной передачи информации об успехе или ошибке вызывающей стороне. Такой подход позволяет ViewModel не знать о деталях источника данных и концентрироваться на подготовке состояния для UI.

Список сообщений отображается на экране «Лента» с помощью RecyclerView. Для него созданы адаптер и ViewHolder, отвечающие за связывание объектов данных с элементами разметки. Дополнительно реализована кнопка «Обновить», инициирующая ручную синхронизацию: ViewModel вызывает метод репозитория, тот выполняет сетевой запрос и сохраняет результат в Room, а Flow в DAO автоматически обновляет список на экране. Если приложение запускается впервые без интернета и локальная база пуста, отображается информационное сообщение о недоступности данных.

Итогом третьей лабораторной работы стало создание полноценного и устойчивого контура данных, обеспечивающего корректную работу приложения как в онлайн-, так и в офлайн-режиме. Единственная точка доступа к данным через репозиторий, реактивное получение обновлений через Flow и разделение сетевого и локального слоев подготовили архитектуру к внедрению фоновой синхронизации на следующем этапе.

4. Улучшение интерфейса и фоновая синхронизация

В рамках четвёртого этапа разработки приложение получило завершённый пользовательский интерфейс, интерактивные элементы и механизм автоматического обновления данных в фоновом режиме. Основная цель данного этапа — привести внешний вид приложения к актуальным стандартам платформы, повысить интерактивность и обеспечить надёжную фоновую синхронизацию с поддержкой системных уведомлений.

Интерфейс экрана «Лента» был переработан с применением компонентов Material Design. Каждый элемент списка сообщений оформлен в виде карточки CardView, содержащей аватар пользователя, его имя и текст сообщения. Аватары реализованы как кастомный элемент разметки внутри RecyclerView. Верхняя часть экрана оформлена с использованием AppBarLayout, обеспечивающего корректное поведение панели заголовка при прокрутке списка. Для быстрого ручного обновления ленты добавлен FloatingActionButton, расположенный в соответствии с рекомендациями Material Design.

В адаптер RecyclerView добавлена поддержка функции «лайк»: каждый элемент списка содержит иконку, меняющую состояние при нажатии. Информация о лайках сохраняется в локальной базе данных Room, что позволяет корректно отображать состояние сообщений после перезапуска приложения или при переходе в офлайн-режим. Обработка нажатий реализована во ViewModel, что сохраняет разделение ответственности между UI и бизнес-логикой в соответствии с принципами MVVM.

Ключевым функциональным улучшением стало внедрение фоновой синхронизации с использованием WorkManager. Был реализован периодический воркер SyncWorker, который с заданным интервалом времени проверяет наличие сети и выполняет обновление списка сообщений через репозиторий. Для задачи установлено ограничение NetworkType.CONNECTED, исключающее запуск синхронизации при

отсутствии интернет-соединения. В случае ошибки сети воркер возвращает `Result.retry()`, обеспечивая повторную попытку при восстановлении доступа к сети.

При успешном завершении синхронизации воркер отправляет системное уведомление с сообщением «Новые данные получены». Для работы с уведомлениями создан отдельный канал (`NotificationChannel`), а при запуске приложения реализован запрос разрешения `POST_NOTIFICATIONS` (обязательного для Android 13 и выше). Уведомления отображаются вне зависимости от того, находится ли приложение на переднем плане или свернуто — логика их показа полностью инкапсулирована внутри воркера и не зависит от состояния UI.

Приложение также адаптировано к изменениям состояния сети. При потере соединения данные продолжают отображаться из локальной базы, а новая синхронизация запускается автоматически при восстановлении доступа к интернету, поскольку `WorkManager` сохраняет отложенные задачи и выполняет их при выполнении установленных ограничений. Такое поведение обеспечивает целостность данных и минимальное участие пользователя в процессе обновления контента.

Таким образом, по завершении четвертой лабораторной работы приложение приобрело законченный облик: привлекательный и понятный интерфейс, реализованный в соответствии с `Material Design`, интерактивные элементы, надежную фоновую синхронизацию и систему уведомлений. Все изменения были внесены с сохранением архитектурных принципов `MVVM` и без нарушения ранее выстроенной структуры проекта.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы было спроектировано и реализовано Android-приложение «Мессенджер», последовательно развивавшееся на протяжении четырех лабораторных работ. Основной целью проекта было освоение актуальных подходов к разработке мобильных приложений под Android и формирование практических навыков работы с архитектурными паттернами, инструментами управления данными и системными сервисами платформы.

На первом этапе была создана базовая структура приложения: единая точка входа в виде MainActivity, навигация через BottomNavigationView и набор фрагментов, соединенных с помощью Navigation Component. Это позволило построить устойчивый каркас с удобным переключением между разделами и заложило основу для дальнейшего расширения функциональности.

Второй этап был посвящен внедрению архитектуры MVVM. Использование ViewModel совместно с LiveData обеспечило реактивное обновление интерфейса, сохранность данных при изменении конфигурации устройства и четкое разделение ответственности между слоем UI и бизнес-логикой. Профиль пользователя и настройки приложения стали полноценно управляемыми через отдельные ViewModel.

Третий этап добавил в приложение полноценный слой данных: сетевое взаимодействие через Retrofit с использованием корутин и локальное хранилище Room. Репозиторий обеспечил единую точку доступа к данным и прозрачную логику переключения между онлайн- и офлайн-источниками. Список сообщений реализован через RecyclerView с поддержкой ручного обновления.

На заключительном четвертом этапе приложение получило визуально заверченный интерфейс на основе Material Design: карточки с аватарами и именами отправителей, FloatingActionButton и AppBarLayout. Была добавлена

функция «лайк», реализована фоновая синхронизация через WorkManager и система push-уведомлений, а также обеспечена корректная реакция приложения на изменения сетевого состояния.

Все поставленные задачи были выполнены в полном объёме. Полученные в ходе работы знания и практические навыки могут быть применены при разработке более сложных мобильных приложений и служат надёжной базой для дальнейшего изучения платформы Android.